J1 point action and canonical encoding

Task: `TASK-20261031-4b6d61`. Joint: `J1_point_action_and_encoding`. Reviewer session: `/root/ncp_j1_point`. Date: 2026-11-01 UTC.

The source statements preserve chronological composition, give the affine normal form on an arbitrary additive commutative group, and require exact generator order for the scalar/point equivalence. The independent integer order controls and 2,115 finite canonical-encoding checks pass. The prescribed zero-generator control falsifies the equivalence when its order hypothesis is removed.

Dimension	Result	Evidence scope
Source fidelity	`faithful_scoped`	Four named source signatures, their definitions, and the pinned encoding helper match the abstract point-action claim and Lemma 1.
Fresh Lean machine status	`not_verified`	No fresh elaborated-type output, named-module build, transitive axiom audit or independent kernel recheck was produced here.
Own-joint review outcome	`inconclusive`	Fresh machine replay is still required for `validated_scoped`; source fidelity and finite controls have been completed.

This is an own-joint review. It supplies no verdict on the shortest-word or commutator-witness joints. The separate order-two control below is included because the precommitted plan explicitly assigns its proves-too-much controls to this task.

Immutable inputs and provenance

Producer: PR #2417, exact commit `f4330e1a29feb8fdea9f1eab7131ae5cde2e2b6f`. Plan and own handoff: PR #2847 at `774a6a86f181740285ed9834a648b40fb9aa8ece`. This branch starts at main `4451a2bf8ff9d171442a10ca33997c58e7362a20`; the producer snapshot was read through Git objects and was not changed.

The claim under review is the plan's expressly scoped claim of chronological actions on additive-group points and canonical cyclic encoding. The source reference is `research/THM_COMMUTATOR_KERNEL1.md`, section 1 and Lemma 1. Its section 1 explicitly distinguishes the abstract formalization from the historical full kernel equalities. These primary sources, plus the pinned implementation dependency, were read before constructing controls; no novelty or literature-wide claim is made. Provenance is `internal` for repository sources and `retrieved` for the GitHub dependency fetch; verified by this task.

Lean is pinned to `leanprover/lean4:v4.35.0`; CryptoLean to `60952c696ded7f3da4f4517006d9af6482662487`; Mathlib to `cfc55fc58714a59dc06e6e79edc6d1d95140d3c2`. The full manifest is retained in `raw/inputs/`.

Source	SHA-256
`formal/CryptoResearch/NCP/PointAction.lean`	`52a2fb6ee809fb5b178ea103d30747e46eb791be0a2658329a0cddda12063aa0`
`formal/CryptoResearch/NCP/CyclicPointAction.lean`	`87beff300f7c01b5350969ebdba59e310066f5d68a8ace2e87e4a66384fbf3b5`
CryptoLean `CryptoLean/Group/CyclicEncoding.lean`	`2af79a2d7b95b2651f8eccb1be8a5bcd2f851aa4d50ad77cebb9790c82c9a787`

`raw/source-manifest.json` binds all eleven snapshots by repository, commit, path, Git blob and SHA-256. `raw/source-integrity.json` records seven matching precommitted SHA-256 values and verifies dependency Git blob `673722ed5e5e413b8741646a71213cf51039e2c6` against the fetched bytes. AffineNormalForm is additionally snapshotted and hashed; the plan did not supply its hash.

Clause-by-clause source-type comparison

These are source signatures, with implicit parameters made explicit in the description. They are not presented as fresh `#print` elaboration output. Exact source bytes are retained in `raw/inputs/`.

Declaration and source	Quantifiers and hypotheses	Conclusion and comparison
`word_point_affine_normal_form`, PointAction lines 34–37	Arbitrary `B : Nat`, `A : Type*`, `[AddCommGroup A]`, `points : Fin B -> A`, word `w`, and point `X : A`.	`wordPointAction points w X = shadowSign w · X + sum_i signedCount w i · points i`. This is Lemma 1's point identity, generalized to arbitrary translation points. Taking `points i = s_i · P` recovers the cyclic bias.
`word_action_encode`, PointAction lines 41–51	In addition, arbitrary `n : Nat`, additive homomorphism `encode : ZMod n ->+ A`, scalars `s`, word `w`, and scalar `x`.	Executing the encoded translations at `encode x` equals encoding the scalar action. Additivity supplies translation and negation preservation. Neither injectivity nor its conclusion is assumed. It proves compatibility, without a reverse scalar-recovery implication.
`word_action_canonical_encode`, CyclicPointAction lines 10–15	`P : A`, `hn : n · P = 0`, scalars `s`, word `w`, scalar `x`.	Specializes compatibility to the constructed `CryptoLean.Cyclic.encode P hn`. The hypothesis guarantees the map is well-defined modulo `n`; it does not imply injectivity.
`canonical_word_action_iff`, CyclicPointAction lines 17–27	`P : A`, `ho : addOrderOf P = n`, scalars `s`, word `w`, and `x y : ZMod n`.	Point action reaches `encode y` iff scalar action reaches `y`. Its proof rewrites by the preceding theorem and applies `encode_injective P ho`. The exact-order hypothesis is used at the only reverse-inference step.

The recursive action processes the head letter first and passes its result to the tail. `wordPointAction_append` states action of `u ++ v` as action of `v` after action of `u`. `AffineNormalForm.wordAction` uses `foldl`, and `signedCount` assigns each occurrence the sign determined by negations to its right. These conventions agree with the source's chronological action.

The pinned helper defines `encode` by `ZMod.lift` of integer multiples of `P`; `encode_intCast` states `encode (z : ZMod n) = z · P`. `encode_injective` uses exact additive order, and `encode_range` identifies the range with `AddSubgroup.zmultiples P`. Thus a canonical cyclic map is supplied, rather than an unexplained embedding. Its range need not be all of the ambient group.

No source signature above assumes a prime modulus, oddness, nonzero translations, positive `B`, or a finite ambient group. None needs those assumptions for this algebraic identity. `B = 0` is allowed; `n = 1` has a singleton scalar domain. At `n = 0`, `ZMod 0` is the integer case and exact additive order zero represents infinite order; that case was inspected at source level, not executed in the finite controls. The historical odd-prime conditions belong to the larger setup and full-kernel statements. The canonical iff compares two scalar values for one fixed word, and does not assert that distinct shadows induce distinct actions.

Controls and their results

Command, run from this worktree:

```
python3 coordination/formal-reviews/ncp-pr2417-20261031/reviews/TASK-20261031-4b6d61/raw/controls.py
```

The script exited 0. Full output is `raw/controls.json`; stderr is empty in `raw/controls.stderr`. These are deterministic Python integer/residue checks of the specified operations, not Lean evaluation or experiment measurements.

Control	Exact result
`A = Int`, `points 0 = 1`, `X = 0`, word `[trans 0, neg]`	Output `-1`; shadow sign `-1`, signed count `-1`.
Same inputs, word `[neg, trans 0]`	Output `+1`; shadow sign `-1`, signed count `+1`.
Empty word	Output `0`; sign `+1`, count `0`.
Canonical map into `ZMod 2`, `P = 1`, `n = 2`	120 cases pass.
Canonical map into `ZMod 5`, `P = 1`, `n = 5`	1,875 cases pass.
Canonical map into `ZMod 6`, `P = 3`, `n = 2`	120 cases pass; image is a proper cyclic subgroup of the ambient group.

Each positive panel covers every word of lengths 0 through 3 over one translation and negation, and every triple `(s, x, y)` modulo `n`. Each case checks the affine scalar formula, point/encoding compatibility and both directions of the point/scalar equality equivalence. The total is 2,115 cases; this count is test coverage, not a performance measurement.

For the prescribed false case, let `A = ZMod 2`, `P = 0`, `n = 2`, `w = []`, `x = 0`, and `y = 1`. Then `2 · P = 0` and both encoded points are zero, while the scalar equality `0 = 1` is false. The order of `P` is **1**, so `addOrderOf P = 2` fails. This is an exact counterexample to deleting `ho`; it is not a counterexample to the submitted theorem. No Lean elaboration failure is claimed, because no attempted Lean application was run.

The plan's second false-extension control also passes: at `n = 2`, `B = 1`, `s = 1`, negation acts trivially on the two points. In `C2 x C2`, the abstract elements `t = (1,0)` and `nu = (0,1)` satisfy `t^2 = nu^2 = (nu*t)^2 = 1`, with `nu` still nonidentity. Their point action has only two images. This rejects extending the historical full action-kernel presentation to order two. It leaves the tested order-two affine and canonical-encoding statements intact and makes no independent J2 verdict.

Concrete curve and parser obligations

The four declarations assume a lawful additive commutative group. Applying them to a named curve and serialized implementation still requires the following individually identifiable bridges:

1. Instantiate `A` as the validated point group of the specified nonsingular curve over the specified field, including its point at infinity and the group operations. 2. Establish `addOrderOf P = n` for that concrete generator, and connect each factor-base point to its scalar encoding. `hn` alone is insufficient for the iff. 3. Prove executable point addition, negation and chronological word execution refine the abstract operations, including exceptional coordinate cases. 4. Prove the decoder accepts the intended canonical field and point encodings, checks curve/subgroup membership as required, handles infinity, and makes serialized equality agree with point equality.

These bridges do not appear in the four cited theorem types. The producer's current scoped claim excludes a concrete elliptic-curve parser refinement, so their absence is a boundary of application rather than a mismatch with that claim.

Remaining verification and publication checks

Fresh Lean replay has not been performed. The host snapshot in `raw/infrastructure.json` showed about 112 GiB available memory, an active Rust build, and an EFS mount whose advertised capacity does not establish local build/cache ownership. No exclusive exact-pin Lean workspace/cache was established for this task; the coordinator explicitly retained the separate machine lane. No heavy build, experiment or cloud job was launched. The next machine check is an isolated explicit build of `CryptoResearch.NCP.PointAction`, `CryptoResearch.NCP.CyclicPointAction`, and `CryptoResearch.NCP.ReachabilityControls` at the producer pins, with fresh named type output and transitive axiom audit for the four declarations. That gate must bind the complete workspace and verifier inputs.

This review used Python 3.9.25 from `/usr/bin/python3` only for standard-library arithmetic and packet preparation. The historical Cursor-specific `/usr/local/bin/python3` path is absent; no harness or package-version claim relies on this interpreter. The cloud-environment status tool and network-policy file were unavailable; ordinary Git and the GitHub connector supplied immutable sources successfully. A stalled local dependency-object read was cancelled, and the pinned dependency was retrieved through the connector with its Git blob independently checked.

The plan's blind shortest-word derivation is assigned to J3. This reviewer read producer source and does not claim to have performed that blind derivation. No sibling report or verdict was read. The independence checker is run

with only this task's attestation; expected missing-J2/J3 report findings are retained in `raw/independence-check.txt`. Full-round composition is the coordinator's task. No ledger records or official scientific statuses are changed; full ledger validation belongs to the integrating review packet.

The diagram and inspected PDF are local derived views of this source/control packet. No shared canonical graph is changed: this review adds coverage and an explicit machine-replay obligation without altering the historical measurements or an official claim status. The write scope permits only this task directory; any central problem-index update or session receipt outside it belongs to the coordinator.

Scope diagram

